

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 from math import sqrt
5
6 class selfAttention(nn.Module):
7     def __init__(self,dim_in,dim_k,dim_v):
8         super().__init__()
9         self.dim_in,self.dim_k,self.dim_v=dim_in,dim_k,dim_v
10        self.W_q=nn.Linear(dim_in,dim_k)
11        self.W_k=nn.Linear(dim_in,dim_k)
12        self.W_v=nn.Linear(dim_in,dim_v)
13        self.norm=1/sqrt(dim_k)
14        self.output=nn.Linear(dim_v,dim_in)
15
16    def forward(self,X,mask=None):
17        # X: B*L*in
18        Q,K,V=self.W_q(X),self.W_k(X),self.W_v(X) # B*L*K/V
19        score=torch.matmul(Q,K.transpose(-2,-1)) *self.norm # B*L*L
20
21        if mask is not None:
22            score=score.masked_fill(mask==0,-1e9)
23
24        weight=F.softmax(score,dim=-1) # B*L*L
25        attention=torch.matmul(weight,V) # B*L*V
26        output=self.output(attention) # B*L*in
27
28        return weight , output
29
30 class MultiHeadAttention(nn.Module):
31     def __init__(self,dim_in,dim_k,dim_v,n_head):
32         super().__init__()
33         self.dim_in,self.dim_k,self.dim_v=dim_in,dim_k,dim_v
34         self.n_head=n_head
35         self.W_q=nn.Linear(dim_in,dim_k)
36         self.W_k=nn.Linear(dim_in,dim_k)
37         self.W_v=nn.Linear(dim_in,dim_v)
38         self.norm=1/sqrt(dim_k/n_head)
39         self.output=nn.Linear(dim_v,dim_in)
40
41    def forward(self,X,mask=None):
42        # X: B*L*in
43        batch,seq_len,embed=X.shape
44        Q,K,V=self.W_q(X),self.W_k(X),self.W_v(X) # B*L*nK/nV
45        # âˆ†âˆ†âˆ†B*n*L*K/V
46        Q=Q.view(batch,seq_len,self.n_head,-1).transpose(1,2)
47        K=K.view(batch,seq_len,self.n_head,-1).transpose(1,2)
48        V=V.view(batch,seq_len,self.n_head,-1).transpose(1,2)
49        score=torch.matmul(Q,K.transpose(-2,-1)) *self.norm # B*n*L*L
50
51        if mask is not None:
52            if mask.dim() == 2: # L*L -> B*n*L*L
53                mask = mask.unsqueeze(0).unsqueeze(0).expand(batch, self.n_head, -1, -1)
54            elif mask.dim() == 3: # B*L*L -> B*n*L*L
55                mask = mask.unsqueeze(1).expand(-1, self.n_head, -1, -1)
56            score = score.masked_fill(mask == 0, -1e9)
57
58
59        weight=F.softmax(score,dim=-1) # B*n*L*L
60        attention=torch.matmul(weight,V) # B*n*L*V
61        output=self.output(attention.transpose(1,2).contiguous().view(batch,seq_len,-1)) # B*L*in
62
63        return weight , output
64
65
66 if __name__ == "__main__":
67     dim_in,dim_k,dim_v=16,8,8
68     n_head=4
69     batch_size=2
70     seq_len=5
71
72     mask=torch.randn(batch_size,seq_len,seq_len)
73     X=torch.randn(batch_size,seq_len,dim_in)
74     Atten=selfAttention(dim_in,dim_k,dim_v)
75     MultiAtten=MultiHeadAttention(dim_in,dim_k,dim_v,n_head)
76     weight,output=Atten(X,mask)
77     print("Input shape:",X.shape)
78     print("Attention weight shape:",weight.shape)
79     print("Output shape",output.shape)

```